

CONTENTS

- Overview
- 1 The model is not the agent
- 2 AGENTS.md: how this project works
- 3 Nesting and precedence
- 4 Agent skills: how to do a kind of task
- 5 MCP: reaching tools and data
- 6 A2A: agents talking to each other
- 7 Subagents: work that will not fit one context
- 8 The five in one picture
- Glossary
- Check yourself
- Where to go next

Five Building Blocks of a Modern AI Agent

AGENTS.md, skills, MCP, A2A and subagents — what each one is for, and why they are not interchangeable.

For engineers building agents who need to know which mechanism solves which problem · 26 July 2026

[Watch the source video](#)

[Read the condensed version](#)

[Read the highlights](#)

BEFORE YOU START

- What a context window is and that it is finite
- Familiarity with an agent calling tools
- Comfort reading Python and JSON

Overview

A capable agent is not a capable model. The model generates text and reasons; on its own it is a conversational partner. Everything that makes it an agent is scaffolding built around it, and five pieces cover most of what a frontier coding agent actually runs on.

They divide cleanly by the problem each solves. Two shape behaviour from the inside: AGENTS.md says how this particular project works, and skills say how to perform a particular kind of task. Two reach outward: MCP connects the agent to tools and data, and A2A connects it to other agents. The fifth, subagents, handles work that will not fit in a single context window.

That division is the useful part, because the five are routinely confused. Knowledge that should be a skill ends up permanently loaded in AGENTS.md. Coordination that wants A2A gets built as an MCP tool. Work that needs a subagent gets crammed into one context until quality collapses. Each mistake has a recognisable symptom, and knowing which piece was designed for which problem is most of the fix.

KEY TAKEAWAYS

- ✓ The model is not the agent — the instruction layer wrapped around it is what makes it one.
- ✓ AGENTS.md is a project-level instruction file, read whenever the agent starts work there.
- ✓ Nested AGENTS.md files stack, with those closer to the working directory taking precedence.
- ✓ A skill is a folder with a SKILL.md whose description tells the agent when to invoke it, so it stays out of context until needed.
- ✓ MCP is a standard interface between an agent and external tools or data, replacing per-target connectors.
- ✓ A2A is a standard for agents to discover and delegate to each other, using a published agent card.
- ✓ MCP is asymmetric — agent to tool. A2A is between peers, each autonomous.
- ✓ Subagents run in their own fresh context and return a result, keeping the parent's context clean and enabling parallel work.

01 The model is not the agent

0:00

A model generates text and reasons. What turns it into an agent is the instruction layer wrapped around it.

Frontier agents plan tasks and write code with minimal human involvement, and it is tempting to attribute that to the model. Inside an agent there is a large language model, and it is doing the text generation and the reasoning. By itself, though, it is just a conversational partner.

What turns it into an agent is everything wrapped around it: the instruction layer that tells it how this project works, what procedures to follow, what it can reach, and when to hand work off.

The distinction is worth being firm about because it predicts where your effort goes. Swapping in a better model improves the reasoning inside each step. It does not tell the agent your build command, does not give it access to your database, and does not stop it filling its context with irrelevant files. Those are all scaffolding problems, and they are the ones that decide whether an agent is useful on your codebase.

The five pieces that follow divide by what they solve. Two shape behaviour from the inside. Two reach outward. One handles work too large for a single context.

KEY POINTS

- The model supplies generation and reasoning, nothing else.
- A model alone is a conversational partner, not an agent.
- The instruction layer around the model is what makes it agentic.
- A better model improves each step; it does not supply project knowledge or access.
- The five pieces split into: shape behaviour, reach outward, and scale beyond one context.

The five, by the problem each solves

Worth memorising in this shape rather than as a list. Nearly every design mistake in this area is reaching for the wrong row.

inside the agent – shapes behaviour

AGENTS.md	how this project works	always loaded
skills	how to do a kind of task	loaded on match

reaching outward

MCP	agent → tools and data	client / server
A2A	agent ↔ agent	peer to peer

beyond one context

subagents	spawn a child with its own context, get a result back
-----------	---

02 AGENTS.md: how this project works

0:40

A Markdown file at the project root that the agent reads whenever it starts work there.

AGENTS.md is a plain Markdown text file that sits at the root of a project. Whenever the agent starts work in that project, it reads whatever the file contains.

What goes in it is project-specific operating knowledge: which commands to run for tests, which coding conventions the codebase uses, setup commands, code style rules, how a pull request title should be formatted. A useful way to think of it is a README written for agents rather than for people.

The agent executes what it finds there when it is contextually relevant. If the file says to run `pnpm test` before committing, the agent runs `pnpm test` before committing.

AGENTS.md was introduced by OpenAI and later contributed to the Agentic AI Foundation, which runs under the Linux Foundation. One wrinkle: some agents use a different filename for the same idea — Claude reads `CLAUDE.md`. Different name, essentially the same mechanism.

The cost to keep in mind is that this file is loaded every time, so it competes directly with the context budget on every session. That gives a clear test for what belongs: if it is not relevant to most work in this project, it does not go here. Build commands and prohibitions qualify. A guide to generating quarterly reports does not — that is a skill, which is the next section.

KEY POINTS

- A Markdown file at the project root, read whenever the agent starts work there.
- Holds commands, conventions, setup steps and formatting rules — a README for agents.
- Instructions are executed when contextually relevant, not merely displayed.
- Introduced by OpenAI, later contributed to the Agentic AI Foundation under the Linux Foundation.
- Some agents use another filename for the same idea, such as CLAUDE.md.
- It is loaded every session, so only broadly relevant content belongs in it.

What earns its place in the file

Note how much of it is prohibitions. The repeated mistakes cost more than the missing facts, and 'do not touch this' is the part nobody thinks to write down until an agent has already touched it.

```
# AGENTS.md
```

Commands

```
test:  pnpm test          run before every commit
build: pnpm build        never call tsc directly
lint:  pnpm lint --fix
```

Conventions

- Conventional Commits for messages: feat|fix|chore(scope): subject
- PR titles match the commit subject.
- Money is integer cents. Never float.

Do not

- Do not edit src/generated/ – it is code-generated.
- Do not add dependencies without asking.
- Do not modify migrations that have already shipped.

The test for what goes in it

The file is paid for on every session, whatever the task. That single fact settles most arguments about what belongs.

in AGENTS.md	somewhere else
the test command	how to write a release note
commit message format	how to build a slide deck
directories not to touch	the full API reference

ask: would I want this in the prompt on a task that has
nothing to do with it? if no, it belongs in a skill.

03 Nesting and precedence

2:02

Several AGENTS.md files can coexist. The one closest to the working directory wins.

AGENTS.md files can be nested. A repository can carry one at the root and others inside sub-projects, each with its own rules. Files closer to the working directory override the earlier ones, because they are applied later.

The mechanism matters most in a monorepo, where a single set of rules across wildly different sub-projects is either too vague to help or wrong for half the tree. Nesting lets the root hold what is genuinely universal — commit conventions, security rules, what is off limits — while each sub-project states its own build and test commands.

Think of it as scope resolution rather than file loading. The agent walks from the root down to where it is working, applying each file in turn, and the nearest statement about any given topic is the one that survives.

Two practical consequences follow. Repeating the root's rules in a child file is not harmless duplication — it is a second copy that will drift, and the drifted copy silently wins for anything under it. And a rule you want genuinely universal must sit at the root and be contradicted nowhere, because there is no mechanism for a parent to lock a value against being overridden below it.

KEY POINTS

- Multiple AGENTS.md files can exist in one repository.
- Files closer to the working directory override those further up.
- Root holds what is universal; sub-projects hold what is local.
- It behaves like scope resolution: nearest statement about a topic wins.
- Duplicating a root rule in a child creates a copy that drifts and then silently wins.
- There is no way to lock a rule against being overridden further down.

A monorepo with three scopes

Working in `services/api/`, the agent applies the root then the api file. The test command is the local one; the commit convention is the root one, since nothing nearer contradicts it.

```
repo/
├── AGENTS.md          conventional commits; never edit generated/
├── services/
│   └── api/
│       └── AGENTS.md  test: pytest -q ; python 3.12 ; ruff
└── web/
    └── AGENTS.md      test: pnpm test ; node 22 ; prettier

# working in services/api/ → root, then api
# test command   : pytest -q          (from api)
# commit format  : conventional      (from root, uncontested)
```

Resolution, as code

The whole rule in six lines. Later files overwrite earlier keys, which is exactly why a duplicated root rule in a child file becomes the one that counts.

```
def resolve(cwd, repo_root):
    rules = {}
    for directory in ancestors(cwd, up_to=repo_root): # root → cwd
        path = directory / "AGENTS.md"
        if path.exists():
            rules.update(parse(path)) # nearer file wins on conflict
    return rules
```

A folder with a SKILL.md whose description tells the agent when to pull it in — and which stays out of context until then.

AGENTS.md covers knowledge the agent needs every time. But some knowledge is needed only sometimes and is not project-specific. Suppose the agent needs to know how to build a PowerPoint deck. Loading all that context on every start would clog the context window for no reason whenever the task has nothing to do with slides.

A skill is a folder. Inside it is a file called SKILL.md, along with whatever scripts and resources the task needs. SKILL.md carries metadata including a description, which effectively says: invoke me when the user wants to do X. If the request matches the description, the agent pulls the skill in. If it does not match, the skill sits out of the way, taking up no context.

Skills are another open standard, supported across multiple agent platforms.

The division against AGENTS.md is clean once stated: AGENTS.md is how this project works; a skill is how to do a specific kind of task. One is bound to a place, the other to an activity — which is also why skills travel between projects and AGENTS.md does not.

The description deserves more care than it usually gets. It is the only part of the skill visible before loading, so it is not documentation — it is the routing key. A skill described vaguely is unreachable no matter how good its instructions are, because nothing will ever match it. Write it as a trigger condition, naming the situations and the words a user would actually say.

KEY POINTS

- A skill is a folder containing SKILL.md plus any scripts and resources.
- The description tells the agent when to invoke the skill.
- Unmatched skills consume no context at all.
- Skills are an open standard supported by multiple platforms.
- AGENTS.md is bound to a project; a skill is bound to an activity and travels.
- The description is a routing key, not documentation — a vague one makes the skill unreachable.

A skill folder

Only the frontmatter is visible until the skill matches. The supporting files are not read until execution actually reaches the step that needs them.

```
skills/slide-deck/
├─ SKILL.md           when to use it + the steps
├─ template.pptx      opened during execution
└─ build_deck.py      run at the final step

# SKILL.md
---
name: slide-deck
description: Build a PowerPoint deck. Use when the user asks for
  slides, a deck, a presentation, or a readout.
---

1. Confirm the audience and the time slot. Ask if not given.
2. Draft an outline, one line per slide. Get it approved.
3. Fill template.pptx, one idea per slide, no paragraphs.
4. Run build_deck.py and report the output path.
```

The description decides everything

Both skills could contain identical steps. Only the second will ever run, because the first contains none of the words a person would use when they need it.

```
# unreachable
description: Presentation utilities.

# reachable
description: Build a PowerPoint deck. Use when the user asks for
  slides, a deck, a presentation, or a readout.
```

Why the split saves the context window

Illustrative sizes, but the shape is what matters: the always-present cost scales with how many skills you have, and the loaded cost scales with what the current task needs.

	in AGENTS.md	as a skill
always in context	~1,400 tokens (the whole guide)	~40 tokens (name + description)
when the task matches	~1,400	~1,400
when it does not match	~1,400 ← waste	0

with 30 skills the difference is the whole budget

05 MCP: reaching tools and data

4:49

An open protocol that wraps any tool or data source in a standard interface, so one connector serves every agent.

Agents need to reach external things — APIs, databases, developer tools, SaaS platforms. The difficulty is that each target has its own interface. Without a standard, every agent would need a custom connector for every external thing it might touch.

MCP, the Model Context Protocol, is an open protocol for connecting AI applications to tools, data sources and workflows. It comes with the idea of an MCP server, which wraps a tool or data source in a standard interface. Any agent that speaks MCP can then talk to that tool.

So if an agent needs to pull something from Notion, or create a Stripe payment link, it speaks MCP to the server, and the server handles the underlying API.

MCP started at Anthropic and is now governed under the Agentic AI Foundation at the Linux Foundation, with broad industry support.

The arithmetic is the argument. Bespoke integration work grows as the number of agents times the number of systems; a protocol makes it grow as the sum. A new agent gets every existing server for free, and a new server is immediately usable by every agent.

One design point that repays attention: an MCP server exposes more than functions. Tools are actions the agent invokes. Resources are data it can read, addressed by URI. Keeping those apart matters, because a server that models everything as a tool forces the agent to call a function to fetch data it could simply have been handed. And since the server is the boundary where a model reaches your systems, it is the right place to enforce scope and permission — the prompt is a request, the server is a guarantee.

KEY POINTS

- Agents must reach APIs, databases, developer tools and SaaS platforms, each with its own interface.
- MCP is an open protocol; an MCP server wraps a target in a standard interface.
- Any MCP-speaking agent can use any MCP server.
- Started at Anthropic, now governed under the Agentic AI Foundation at the Linux Foundation.
- Integration effort goes from agents × systems to agents + systems.
- Tools are actions, resources are readable data — modelling everything as a tool is a common error.
- The server is where scope and permission should be enforced.

Why a protocol instead of connectors

Each line on the left is bespoke code somebody maintains. On the right, each server is written once and every current and future agent can use it.

without a standard	with MCP
agent A → Notion	agent A └─┐
agent A → Stripe	agent B ─┴─> MCP → Notion
agent A → Postgres	agent C └─┐ → Stripe
agent B → Notion	→ Postgres
agent B → Stripe	
...	
N × M connectors	N + M

Actions and readable data are different things

Sketched for readability rather than as any specific SDK. The distinction is not stylistic: one has consequences and belongs behind a permission check, the other does not.

```
# TOOL – an action, with consequences
@tool("create_payment_link",
      description="Create a Stripe payment link. Use when asked to bill.")
def create_payment_link(amount_cents: int, currency: str, *, caller) -> str:
    if not caller.has_scope("payments:write"):
        return "REFUSED: this connection is read-only"
    return stripe.PaymentLink.create(amount=amount_cents, currency=currency).url

# RESOURCE – data the agent can simply read
@resource("notion://pages/recent")
def recent_pages() -> str:
    return format_list(notion.search(sort="last_edited", limit=20))
```

06 A2A: agents talking to each other

6:17

An open protocol for agent-to-agent communication, where each agent publishes a card describing what it does and how to reach it.

MCP covers agents talking to tools and data. A2A — agent to agent — is an open protocol for agents talking to each other.

Take a procurement agent that handles vendor contracts and a finance agent that approves spend. The procurement agent negotiates a contract and then needs to hand off to finance for approval. Without A2A the two would need some form of custom integration, or they would not coordinate well at all.

With A2A, each agent publishes an agent card: a description of what the agent does and how to talk to it. Other agents read that card and work out how to delegate. The procurement agent finds and reads the finance agent's card, then hands over the contract.

A2A comes from Google and is now an open standard under the Linux Foundation.

The difference from MCP is worth stating precisely, because the two get conflated. MCP is asymmetric: a client calls a server, the server exposes a fixed surface, and nothing on the far side is making decisions. A2A is between peers: both sides are autonomous agents, either can initiate, and the receiving agent decides how to accomplish what it was asked rather than executing a named function.

That difference has a practical edge. An MCP tool call either succeeds or fails, quickly and deterministically. A delegated A2A task is a long-running, stateful thing another autonomous system is working on, which means you need a task identity, a way to poll or subscribe to progress, and an answer to

what happens when the other agent decides it cannot do it. Treating delegation like a function call is the mistake to avoid.

KEY POINTS

- A2A is an open protocol for communication between agents.
- Each agent publishes an agent card describing what it does and how to reach it.
- Other agents read the card to work out how to delegate.
- It comes from Google and is now an open standard under the Linux Foundation.
- MCP is asymmetric client-to-server; A2A is between autonomous peers.
- An MCP call returns; an A2A delegation is a long-running task needing identity and progress.
- The receiving agent decides how to accomplish the request, not which function to run.

An agent card

Illustrative shape rather than an exact schema — the point is what a card has to carry. Discovery needs a stable location, delegation needs an endpoint and auth, and routing needs a description specific enough to choose on.

```
{
  "name": "finance-approvals",
  "description": "Approves vendor spend against policy and budget.",
  "skills": [
    {"id": "approve_contract",
     "description": "Review a vendor contract and approve or reject spend."},
    {"id": "check_budget",
     "description": "Report remaining budget for a cost centre."}
  ],
  "endpoint": "https://finance.internal/a2a",
  "auth": {"type": "oauth2", "scopes": ["spend:approve"]}
}
```

published at a well-known location so other agents can find it

MCP and A2A solve different shapes of problem

The row that decides which you need is the last one. If the far side merely executes what you named, it is a tool. If it works out how, it is an agent.

	MCP	A2A
relationship	client → server	peer ↔ peer
far side is	a tool or data source	another agent
who initiates	the agent, always	either side
you specify	which function to run	what outcome you want
duration	a call	a task, possibly long
failure	an error you handle	a peer that declined
the far side	executes	decides how

Delegation is not a function call

The shape that gets this wrong in production is awaiting a delegation as though it were a tool call. A peer may take minutes, ask for clarification, or decline — none of which a function does.

```
card = discover("finance-approvals")

task = a2a.delegate(
    card, skill="approve_contract",
    payload={"vendor": "Acme", "annual_eur": 48_000, "doc": contract_uri},
)

while True:
    state = a2a.poll(task.id)
    if state.status == "completed":
        return state.result
    if state.status == "rejected":
        return escalate_to_human(state.reason)
    if state.status == "input-required":
        a2a.reply(task.id, answer_for(state.question))
    sleep(state.retry_after)
```

07 Subagents: work that will not fit one context

8:24

A child agent spawned with its own fresh context, which does a piece of work and

| returns a result.

Sometimes one agent is not enough, and there are two distinct reasons.

The first is size. An agent reviewing a codebase with thousands of files would blow out its context just loading them. The second is parallelism. If the work is embarrassingly parallel — twenty independent checks on twenty functions — doing them one at a time is slow, when doing them at once would be twenty times faster.

A subagent is a child agent that the main agent spawns to do a specific piece of work. Each runs in its own fresh context window, does its job, and returns a result when done. The parent can send a child to read five hundred files and hand back a summary, keeping the parent's own context clean. Or it can run twenty children in parallel on twenty independent checks.

Subagents differ from the other four in one respect: they are a common pattern in modern agent systems rather than a formal standard with a specification behind them. The concept nevertheless shows up almost identically everywhere — a parent spawns children, each does its work, each returns a result, and the parent continues with its context intact.

The property doing the work is context isolation. The child reads five hundred files; the parent receives a paragraph. That compression is the entire benefit, and it is also the constraint: whatever the child does not put in its summary is gone. So the instruction you give a child matters more than usual, because it determines what survives.

And the cost is real. Each child is a full agent with its own model calls, so twenty children is roughly twenty times the token spend, bought to save wall-clock time. Fan out when the work is genuinely independent and the results compress well. Do not fan out when the subtasks need to see each other's findings — coordinating through a parent that only has summaries is usually worse than one agent doing the work in sequence.

KEY POINTS

- Two motivations: work too large for one context, and work that is embarrassingly parallel.
- A subagent runs in its own fresh context, does a job and returns a result.
- The parent's context stays clean because it receives only the result.
- Not a formal standard, but near-identical across modern agent systems.
- Whatever the child leaves out of its summary is lost to the parent.
- Cost scales with the number of children; you are buying latency with tokens.
- Do not fan out when subtasks need each other's findings.

Fan out, then aggregate

The parent never sees the files. It sees twenty short reports, which is what keeps its context usable. Note that a failing child returns a value rather than raising — one bad check should not lose the other nineteen.

```
def review(functions, parent_ctx):
    reports = run_parallel([
        spawn_subagent(
            task=f"Review {fn.path}:{fn.name} for error handling only. "
                f"Reply with at most 5 lines: findings or 'none'.",
            context="fresh",
        )
        for fn in functions
    ], on_error=lambda fn, e: f"{fn.name}: FAILED ({e})")

    return summarise(reports)      # parent sees ~20 short reports,
                                   # never the 20 files
```

The instruction determines what survives

The child's context is discarded when it finishes, so anything not in the reply is unrecoverable. A vague instruction produces a vague summary and the parent has no way to ask again without redoing the work.

```
# weak – the parent gets prose and cannot act on it
"Read the auth module and tell me about it."

# strong – the parent gets something it can use
"Read src/auth/. Return exactly:
  - every function that catches an exception without re-raising
  - file:line for each
  - one sentence on what failure it hides
Nothing else. If there are none, reply 'none'."

# whatever is not in the reply is gone when the child exits
```


When not to fan out

The left column is where subagents pay. The right column is where they cost more than they save, because the parent ends up mediating between children that needed to talk to each other.

fan out	keep it sequential
20 independent lint checks	a refactor spanning 3 modules
summarise 500 files	a debug hunt following one thread
search a monorepo	work where step 2 needs step 1's
test N configurations	actual output, not a summary

08 The five in one picture

10:28

Two shape behaviour, two reach outward, one scales past a single context — and the symptoms tell you which is missing.

Put them together and the architecture of a frontier agent is legible. AGENTS.md and skills live inside the agent and shape how it behaves. MCP and A2A are how it reaches outward, to tools and to other agents. Subagents handle work that does not fit in one context.

The reason to hold the map rather than the list is that these get substituted for one another, and each substitution has a recognisable symptom.

Procedural knowledge stuffed into AGENTS.md burns context on every session for something needed occasionally — the symptom is a context budget mostly spent before work starts. Coordination built as an MCP tool when it wanted A2A produces a tool that is really a request to another autonomous system, and it breaks the first time that system needs to ask a clarifying question. Work crammed into one context that needed subagents shows up as quality degrading through a long task as earlier material gets crowded out.

The diagnostic habit worth adopting: when an agent underperforms, ask which of the five is missing before asking whether the model is good enough. The answer is far more often the scaffolding, and the scaffolding is what you can actually change.

KEY POINTS

- AGENTS.md and skills shape behaviour from inside the agent.
- MCP reaches tools and data; A2A reaches other agents.
- Subagents handle work that will not fit one context window.
- Procedural knowledge in AGENTS.md shows up as a context budget spent before work begins.
- Coordination built as an MCP tool breaks when the far side needs to ask something.
- Diagnose the missing piece before questioning the model.

Symptom to missing piece

Each symptom points at exactly one of the five, and the fixes are not interchangeable. Note that none of the rows is answered by a better model.

symptom	what is missing
"it uses the wrong test command every time"	AGENTS.md
"it knows the facts, botches the procedure"	a skill
"it cannot reach our database at all"	an MCP server
"two agents duplicate each other's work"	A2A
"quality drops halfway through long tasks"	subagents
"the context is full before it starts"	a skill (content is in AGENTS.md)

The whole stack, assembled

One turn, all five in their places. Most turns touch only the first two — which is the point of loading the rest on demand.

start of session	
read AGENTS.md (root → cwd)	always
list skill names + descriptions	always, ~40 tokens each
on a task	
match a skill? load its SKILL.md	on demand
need a tool? call it over MCP	on demand
need a peer? read its agent card,	
delegate over A2A	on demand
too big / parallel? spawn subagents,	
collect results	on demand

Glossary

AGENTS.md

A Markdown file at a project root holding commands, conventions and prohibitions, read whenever an agent starts work in that project.

Agentic AI Foundation

The body under the Linux Foundation that now governs several of these standards, including AGENTS.md and MCP.

Nesting / precedence

Several AGENTS.md files applied from the root down, with the file closest to the working directory overriding those above it.

Agent skill

A folder containing a SKILL.md plus scripts and resources, describing how to perform one kind of task.

SKILL.md

The file defining a skill: metadata including the description that tells the agent when to invoke it, and the steps to follow.

Skill description

The one visible part of a skill before loading. It functions as a routing key, so a vague description makes the skill unreachable.

MCP

Model Context Protocol: an open protocol connecting AI applications to tools, data sources and workflows through a standard interface.

MCP server

The component wrapping a tool or data source so any MCP-speaking agent can use it, and the right place to enforce permission.

A2A

Agent to agent: an open protocol letting autonomous agents discover one another and delegate work.

Agent card

A published description of what an agent does and how to talk to it, which other agents read in order to delegate.

Subagent

A child agent spawned by a parent, running in its own fresh context, doing one piece of work and returning a result.

Context isolation

The property that makes subagents useful: the child holds the bulk of the material and the parent receives only the conclusion.

Embarrassingly parallel

Work that splits into independent pieces needing no coordination — the case where fanning out to subagents pays.

Check yourself

Answer first, then open each one.

A guide for generating quarterly reports is added to AGENTS.md and the agent starts running out of context on unrelated tasks. What went wrong?

AGENTS.md is loaded on every session regardless of the task, so occasional, activity-specific knowledge placed there is paid for constantly. It belongs in a skill, which stays out of context — costing only its description — until a request matches it.

A monorepo root AGENTS.md says to use conventional commits, and a sub-project file repeats that rule with slightly different wording. Why is this worse than not repeating it?

Files closer to the working directory override those above, so the duplicated copy is the one that applies under that sub-project. It will drift from the root over time and silently win, producing behaviour that contradicts the rule everyone believes is in force — and there is no way for the root to lock a value against being overridden.

A skill has excellent instructions and is never invoked. Where is the fault almost certainly?

In the description. It is the only part visible before loading, so it acts as the routing key. If it does not name the situation and the words a user would actually say, nothing matches it and the instructions are never read — the decision to load happens before they are visible.

Why does MCP change integration effort from a product into a sum?

Without a standard, each agent-to-system pairing needs its own connector, so the work scales as agents times systems. With MCP each system needs one server and each agent speaks the protocol once, so it scales as agents plus systems — and any new agent immediately gets every existing server.

What is the essential difference between calling an MCP tool and delegating over A2A?

An MCP call is asymmetric and specifies which function to run; the far side executes and returns. An A2A delegation goes to an autonomous peer that decides how to achieve the outcome, may take a long time, may ask a clarifying question, and may decline. Treating delegation as a function call breaks the first time any of those happens.

You spawn twenty subagents to review twenty files and get back twenty vague paragraphs. Why is this unrecoverable without redoing the work?

Each child's context is discarded when it finishes, so anything not in its reply is gone — the parent never saw the files and cannot query what the child read. The instruction given to a child therefore determines what survives, which is why it should specify the exact shape of the reply rather than asking for a general account.

Where to go next

- Audit your own AGENTS.md and move anything task-specific into a skill, then measure how much of the context budget you get back on an unrelated task.
- Set up a nested AGENTS.md in a monorepo and deliberately create a conflicting rule, to see for yourself which one the agent follows.
- Rewrite the descriptions of skills that never trigger as explicit trigger conditions naming user phrasings, then count how often they are selected afterwards.
- Write an MCP server for one internal system and notice how much of the work is deciding what should be a tool versus a resource rather than writing the integration itself.
- Model one hand-off between two of your services as an A2A delegation and enumerate the states you have to handle — completed, rejected, input-required — that a function call never has.
- Take a long single-agent task where quality degrades and split the bulk reading into a subagent, then compare the parent's context usage before and after.
- The MCP entry here has a fuller treatment elsewhere in this collection, and the memory lesson covers how AGENTS.md and skills map onto semantic and procedural memory — this page is the map, those are the depth.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.